Python 실습

계산기 CLI 만들기

cchyun@gmail.com

Part 1. 실습 설계 및 요구사항

계산기 CLI — 왜 만드나?

- **로봇 SW의 핵심 구조: 인지 → 판단 → 동작(Sense-Think-Act)의 반복**
  - 계산기 프로그램은 이 **제어 루프(Control Loop)** 의 가장 단순한 형태
    - 사용자 입력을 받고(Sense)
    - 연산자에 따라 판단하며(Think)
    - 결과를 출력(Act)
  - 오늘 배운 기초 문법들이 **실제 로봇의 '지능'을 구현하는 핵심 도구**
  - **복잡한 프로그램을 한 번에 만들지 않고, 기능을 하나씩 추가**하는 점진적 구현 방식 채택

프로그램 요구사항

- **계산기 CLI 최종 목표**
  - **입력:** 사용자로부터 두 개의 숫자와 한 개의 연산자(`+`, `-`, `*`, `/`)를 입력받음
  - **연산:** 선택한 연산자에 따라 사칙연산 수행
  - **반복:** 사용자가 종료를 원할 때까지 계산 과정을 무한히 반복
    - 로봇의 메인 루프(Main Loop)와 동일한 구조
  - **예외 처리:** 숫자가 아닌 값 입력, 0으로 나누기 등 비정상 상황을 안전하게 처리
  - **구조화:** 각 연산 로직을 함수로 분리하여 코드의 재사용성 향상

단계별 구현 순서 — 점진적 개발

- **복잡한 기능을 한 번에 구현하지 않는다 — 기능 하나씩 완성하며 쌓아가기**
  - **1단계**: 기본 입력과 출력
    - `input()`, `float()`, `print()` — **자료형 학습**
  - **2단계**: 연산자 선택 기능 추가
    - `if-elif-else` — **제어문(조건문) 학습**
  - **3단계**: 무한 루프 구현
    - `while True`, `break` — **제어문(반복문) 학습**
  - **4단계**: 함수를 이용한 모듈화
    - `def`, `return` — **함수 학습**
  - **5단계**: 비정상 입력 처리 (최종 완성본)
    - `try-except` — **예외 처리 학습**

Part 2. 단계별 구현

1단계 — 기본 입력과 출력

- **사용자로부터 두 숫자를 받아 더한 결과를 출력**

  - `input()` 은 모든 입력을 **문자열(str)** 로 반환
  - 산술 연산을 위해 **실수형(float)** 으로 형변환 필수

```
# 1단계: 두 숫자를 입력받아 더하기
num1 = float(input("첫 번째 숫자를 입력하세요: "))
num2 = float(input("두 번째 숫자를 입력하세요: "))

result = num1 + num2
print(f"결과: {num1} + {num2} = {result}")
```

- **핵심 포인트**

  - `float(input(...))` — 입력과 형변환을 한 줄에 처리
  - f-string `f"..."` — 변수를 문자열 안에 삽입

2단계 — 연산자 선택 기능 추가

- `if-elif-else` 구조로 사용자가 원하는 연산을 선택

```
# 2단계: 조건문을 활용한 사칙연산 선택
num1 = float(input("첫 번째 숫자: "))
num2 = float(input("두 번째 숫자: "))
op = input("연산자(+, -, *, /)를 선택하세요: ")

if op == '+':
    print(f"결과: {num1 + num2}")
elif op == '-':
    print(f"결과: {num1 - num2}")
elif op == '*':
    print(f"결과: {num1 * num2}")
elif op == '/':
    print(f"결과: {num1 / num2}")
else:
    print("잘못된 연산자입니다.")  # 예외 상황 처리
```

- **핵심 포인트**
  - `if-elif-else`의 마지막 `else`는 **예외 상황을 반드시 처리**
  - 로봇 안전 로직에서 "만약 거리 < 0.5m이면 정지" 구조와 동일

3단계 — 무한 루프 구현

- **로봇이 전원이 꺼질 때까지 작동하듯, 사용자가 끝낼 때까지 반복**

```
# 3단계: while 루프를 이용한 반복 실행
while True:
    print("\n--- 로봇 계산기 가동 중 ---")
    op = input("연산자(+, -, *, /) 또는 'q'를 입력하여 종료: ")

    if op == 'q':
        print("프로그램을 종료합니다.")
        break  # break는 루프를 즉시 탈출

    num1 = float(input("첫 번째 숫자: "))
    num2 = float(input("두 번째 숫자: "))

    if op == '+':
        print(f"결과: {num1 + num2}")
    # ... (나머지 연산 로직)
```

- **핵심 포인트**
  - `while True` — 로봇 **메인 운영 루프**의 기본 형태
  - `break` — 종료 조건을 만족할 때 루프 탈출

4단계 — 함수를 이용한 모듈화

- **코드의 가독성과 유지보수를 위해 연산 로직을 함수로 분리** (1/2)

```
# 4단계: 함수(Function) 정의 및 호출
def add(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): return a * b
def divide(a, b):
    if b == 0:
        return "Error: 0으로 나눌 수 없습니다."
    return a / b

while True:
    op = input("연산 선택(+, -, *, /) / 종료(q): ")
    if op == 'q': break

    n1 = float(input("숫자 1: "))
    n2 = float(input("숫자 2: "))

    if op == '+': print(add(n1, n2))
    elif op == '-': print(subtract(n1, n2))
    # ... (나머지 호출)
```

4단계 — 함수를 이용한 모듈화

- **코드의 가독성과 유지보수를 위해 연산 로직을 함수로 분리** (2/2)
- **핵심 포인트**
  - 함수로 분리 → 나중에 필요할 때마다 **재사용** 가능
  - `divide()` 안의 `if b == 0` — 함수 내부에서 선제적 예외 처리

5단계 — 비정상 입력 처리 (1/2)

- **사용자가 숫자가 아닌 문자를 입력해도 프로그램이 강제 종료되지 않도록**

```
# 5단계: try-except를 이용한 견고한 코드 만들기
def calculator():
    """로봇 SW 개발 입문용 계산기 메인 함수"""
    while True:
        try:
            print("\n[메뉴] +, -, *, / 중 하나를 선택하세요 (종료: q)")
            op = input("선택: ").strip()

            if op.lower() == 'q':
                break

            if op not in ['+', '-', '*', '/']:
                print("⚠️ 잘못된 연산자입니다. 다시 입력해주세요.")
                continue  # 루프의 다음 회차로 넘어감

            num1 = float(input("첫 번째 숫자 입력: "))
            num2 = float(input("두 번째 숫자 입력: "))
```

5단계 — 비정상 입력 처리 (2/2)

```
            if op == '+': print(f"결과: {num1 + num2}")
            elif op == '-': print(f"결과: {num1 - num2}")
            elif op == '*': print(f"결과: {num1 * num2}")
            elif op == '/':
                if num2 == 0:
                    raise ZeroDivisionError  # 인위적으로 에러 발생
                print(f"결과: {num1 / num2}")

        except ValueError:
            print("⚠️ 에러: 유효한 숫자를 입력해야 합니다.")
        except ZeroDivisionError:
            print("⚠️ 에러: 0으로 나눌 수 없습니다.")
        except Exception as e:
            print(f"⚠️ 알 수 없는 에러 발생: {e}")

if __name__ == "__main__":
    calculator()
```

- **핵심 포인트**
  - `continue` — 루프의 다음 회차로 즉시 넘어감
  - `raise` — 조건에 맞을 때 직접 예외를 발생시킴
  - `if __name__ == "__main__"` — 모듈로 import될 때와 직접 실행될 때를 구분

Part 3. 오류 해결 및 심화 미션

자주 발생하는 오류와 해결법

오류 메시지	원인	해결 방법
ValueError: could not convert string to float	`input()` 으로 받은 값에 숫자 이외 문자 포함	`float()` 변환 전 `try-except` 로 감싸거나 입력값 검증
IndentationError: expected an indented block	Python 들여쓰기 규칙 위반	`if`, `while`, `def` 다음 줄은 반드시 4칸 공백 사용
ZeroDivisionError: float division by zero	분모에 0이 들어간 상태로 나눗셈 수행	나눗셈 전 `if num2 == 0:` 으로 체크하거나 예외 처리
NameError: name '...' is not defined	변수나 함수를 선언 전에 사용	오타 확인, 함수 정의가 호출 위에 있는지 확인

심화 미션 (빠른 학습자용)

- **기본 실습을 완료했다면 도전해보자**

  - **미션 1: 계산 기록 저장 (List 활용)**
    - 수행한 모든 계산 결과(예: `10 + 5 = 15`)를 리스트에 저장
    - 사용자가 `'history'`를 입력하면 모든 기록을 출력

  - **미션 2: 고급 수학 연산 (math 모듈)**
    - Python 내장 `math` 모듈을 `import`하여 기능 추가
    - 거듭제곱(`pow`), 제곱근(`sqrt`), 절대값(`abs`) 연산 지원

  - **미션 3: 정규표현식 입력 검증 (re 모듈)**
    - `re` 모듈을 사용하여 `숫자 연산자 숫자` 형태(예: `10 + 20`) 검증
    - 한 번에 식을 입력받아 파싱하는 로직 구현

Part 4. 로봇 SW와의 연결

계산기 개념 → 로봇 코드 활용

- **로봇은 '인지-판단-동작(Sense-Think-Act)'의 순환 구조로 작동**
  - **함수(Function):** 계산기의 `add()`, `divide()` 처럼 로봇도 복잡한 동작을 모듈화
    - 좌표 변환 함수, PID 제어 계산 함수 → 필요할 때마다 재사용
  - `if` **문:** 로봇 **안전 로직**의 핵심
    - "만약 장애물과의 거리가 0.5m 미만이면 정지하라"
  - `while` **문:** 로봇이 전원이 꺼질 때까지 계속 작동하는 **메인 운영 루프**
  - `for` **문:** Lidar 센서의 수백 개 거리 정보 중 **가장 가까운 장애물 탐색**
  - **예외 처리:** 센서 연결 끊김, 값 튀는 현상 등 **불확실한 하드웨어 환경에서 복원력 유지**

센서 데이터 처리 CLI로의 확장

- **계산기가 숫자 입력을 기다리듯, 로봇 CLI는 센서 값을 입력으로 받음**
  - `input("두 번째 숫자: ")` → `get_sensor_data()` 함수로 거리 센서 값 수신

```
# 장애물 판단 로직 예시
distance = float(input("센서 거리 입력(m): "))
if distance < 1.0:
    print("[경고] 장애물 감지! 회전합니다.")  # 'Act' 단계의 시작
else:
    print("[안전] 직진 중...")
```

- **이 간단한 CLI 로직이 발전하면 → 실제 로봇 장애물 회피 알고리즘**
  - Turtlebot3가 센서 데이터를 구독(Subscribe)하고
  - 장애물을 피해 이동하는 **'Wander' 알고리즘**의 기초

Day 2 연결고리 — OOP와 NumPy

- **오늘 배운 기초는 내일 배울 심화 주제를 위한 징검다리**

  - **객체 지향 프로그래밍(OOP)**
    - 지금은 기능별로 함수를 만들었지만, 내일은 이를 묶어 **'로봇 구성 요소'** 단위로 설계
    - `Motor` 클래스 안에 속도 변수와 회전 함수를 묶어 관리
    - 실제 로봇 하드웨어의 구조를 코드로 그대로 옮겨오는 법 학습

  - **NumPy**
    - 오늘 리스트(List)에 데이터를 담아봤지만 → 로봇 데이터(수천 개의 Lidar 포인트)엔 너무 느림
    - NumPy는 대량의 수치 데이터를 **고속으로 계산** → 로봇이 실시간으로 환경 파악 가능

마무리 — 오늘의 핵심 정리

- 계산기 CLI = 로봇 제어 루프의 가장 단순한 축약 모델
  - 외부 정보를 받아(Input) → `input()`, `float()`
  - 수학적 논리로 판단하고(Process) → `if-elif-else`, 함수
  - 결과를 출력(Output) → `print()`, f-string
  - 오류가 생겨도 멈추지 않고(Robustness) → `try-except`
  - 이 기초가 탄탄해야 복잡한 자율 주행 로직도 구현할 수 있다